

Assignment Category: CAT_12

Project Name: StudyNook – Library Study Room Booking

Requirement Video:  StudyNook Requirments Video.mp4

Project Theme

StudyNook is a full-stack web application where students and library users can list study rooms they control (e.g., private rooms in a university library), and any registered user can browse, search, filter, and book those rooms for a specific date and time slot. The platform automatically prevents double-booking using time-conflict detection, allows room owners to manage their listings, and gives every user a dashboard to handle their own bookings. Secure JWT authentication is stored in HTTP-only cookies, and the whole site is responsive and recruiter-friendly.

Key Rules:

- **GitHub Commits:**
 - Include a minimum of 15 notable GitHub commits on the client side.
 - Include a minimum of 8 notable GitHub commits on the server side
- **Readme.md:** Add a meaningful readme.md file with the name of your website and a live site URL on client side. Include a minimum of five bullet points to feature your website.
- **Lorem Text:** Don't use any Lorem ipsum text; you can not use the default alert to show any error or success message.
- **Host your Application:** You can choose deployment systems like Vercel, Render for hosting. As you develop a single-page application
 - Ensure that the page doesn't throw any error on reloading from any routes.
 - Logged in User must not be redirected to Login on reloading any private route

- **Environment Variables:** Keep MongoDB connection string, JWT secret, and any API keys in .env files (securely hidden).
 - **Design:** Completely unique; not similar to any module example or previous assignment. Must be responsive for mobile, tablet, and desktop.
-

1. Layout Structure

Public Layout (Not logged in)

- **Navbar:**
 - Logo / website name
 - Navigation links: Home, Rooms (All Rooms)
 - Login button
 - Register button
- **Footer:**
 - Useful links (Home, Rooms, About)
 - Contact information (email, phone)
 - Social icons (Facebook, X (new logo), LinkedIn, Instagram)
 - Copyright text

Private Layout (Logged in)

- **Navbar:**
 - Same logo and Home, Rooms
 - Additional links: Add Room, My Listings, My Bookings (show when user login)
 - Profile dropdown (photo thumbnail + name, or you can put My Listings, My Bookings also) with Logout
 - **Footer:** Same as public
 - **Private Routes** (Add Room, My Listings, My Bookings, Room Details “Book Now” action)
-

2. Home Page (/)

Banner Section

- **A hero area containing:**
 - Title: e.g., “Find Your Perfect Study Room”

- Description: “Browse and book quiet, private study rooms in your library. List your own room and earn.”
- A prominent Explore Rooms button → links to /rooms

Dynamic Section – Available Study Rooms

- Use MongoDB’s `sort()` and `limit()` methods to retrieve only the latest 6 rooms from the database.
- Each room is displayed as a card with equal height/width (grid layout: 3 columns desktop, 2 tablet, 1 mobile).
- Card contents:
 - Room image (uniform size, object-fit: cover)
 - Room name
 - Short description (truncated to ~100 characters)
 - Floor (e.g., “Floor 3”)
 - Seat capacity (e.g., “2–4 people”)
 - Hourly rate (e.g., “\$5/hr”)
 - Amenities shown as small chips (max 3, rest as “+X more”)
 - “View Details” button → redirects to /rooms/:id

Two Extra Static Sections

- Implement any **two additional sections** to enhance the homepage experience
-

3. Authentication

User Login (/login)

- Form fields:
 - Email (type email, required)
 - Password (type password, required)
- Buttons:
 - Login (submit)
 - Continue with Google (Google OAuth)
- On successful login (email/password or Google):
 - Redirect to the home page (or the route the user intended to visit before being asked to log in).
- On failure:
 - Show an error toast (e.g., “Invalid email or password”) or inline message inside the form – no alert().

- Link to Register page: “Don’t have an account? Register”

User Registration (/register)

- Form fields:
 - Name (text, required)
 - Email (email, required)
 - Photo URL (text, required – user must provide an image URL; you may also later upload but URL is simpler)
 - Password (password, required)
- Password validation (must be checked before submission):
 - At least 6 characters
 - At least one uppercase letter
 - At least one lowercase letter
 - If criteria fail, show an inline error message and prevent the form from submitting.
- Buttons:
 - Register
 - Continue with Google
- On successful registration (email/password):
 - Store user in DB, then redirect to Login page.
 - Show a success toast “Registration successful! Please login.”
- On Google registration: user is logged in directly and redirected to Home.
- Link to Login page: “Already have an account? Login”

💡 Don’t implement email verification or the forget password method, as it will inconvenience the examiner. If you want, you can add these after receiving the assignment result.

4. CRUD Operations – Study Rooms

All CRUD operations manipulate the rooms collection. Any authenticated user can create a room; they become the room’s owner. Users can only edit or delete their own rooms (checked on the server by comparing req.user.id with the room’s owner field).

4.1 Add Room (Private Route /add-room)

- Form fields:
 - Room Name (text, required)

- Description (textarea, required)
- Image (image URL from internet)
- Floor (text/number, e.g., “3rd Floor”)
- Capacity (number, e.g., 4)
- Hourly Rate (number, e.g., 5 – in dollars)
- Amenities – a set of checkboxes. Options: Whiteboard, Projector, Wi-Fi, Power Outlets, Quiet Zone, Air Conditioning. Selected values are joined into an array of strings.
- On success: toast “Room added successfully” and redirect to My Listings or Rooms page.

4.2 All Rooms Page (/rooms)

- Public route. Fetches all rooms from the DB.
- Display in the same card grid as the home page (keeping uniform image size).
- If no rooms match, display a friendly “No rooms found” message.

4.3 Room Details Page (/rooms/:id)

- Full details displayed:
 - Booking Count – a number that increments every time the room is booked (see challenge section)
 - “Book Now” button:
 - If user is logged in → opens the booking form (modal or inline).
 - If not logged in → button reads “Login to Book” and redirects to Login.
- Edit & Delete controls (only visible if current user is the room owner):
 - Edit button opens a pre-filled form (modal or separate page) where all fields can be updated.
 - Delete button triggers a confirmation modal; after confirmation the room is permanently deleted.

4.4 Update Room (Owner only)

- API: PUT /api/rooms/:id (or PATCH)
 - Verifies owner matches req.user.id.
 - Accepts the updated fields (partial or full). Image URL can be changed by uploading a new picture (optional; if not changed, keep the existing URL).
- Success toast: “Room updated successfully”.

4.5 Delete Room (Owner only)

- API: DELETE /api/rooms/:id
 - Server checks ownership.
 - Before deleting, optionally use \$pull to remove the room's ID from any user's bookings array (if any bookings reference this room).
 - Delete the room document.
 - Success toast: "Room deleted successfully".
-

5. Booking System

All bookings are stored in a bookings collection.

5.1 Book a Room (Private)

- Triggered by "Book Now" on the room details page.
- Booking form (modal or inline) contains:
 - Date – date picker; must be today or a future date.
 - Start Time – dropdown with hourly slots: 08:00, 09:00, ..., 20:00.
 - End Time – dropdown, only times after the selected start time (same hour granularity). Minimum booking is 1 hour.
 - Total Cost – automatically computed in real-time: (endHour - startHour) * hourlyRate.
 - Special Note (optional text).
 - Confirm Booking button.
- Conflict check:
 - Check if the selected room and time slot already has a confirmed booking.
 - If conflict exists, return an error; otherwise, create the booking successfully.
- Frontend shows a success toast "Room booked successfully!" and closes the modal.

5.2 My Bookings Page (Private /my-bookings)

- Fetch bookings where userId matches the logged-in user, populated with room data (name, image).
- Display in a table or card list.:
 - Status badge: "confirmed" (green) or "cancelled" (red)
 - Cancel button – visible only if status is "confirmed" and the booking date is in the future (today's future or later).

- Empty state: “You have no bookings yet.”

5.3 Cancel Booking (Private)

- Clicking “Cancel” opens a confirmation modal.
 - On confirm, send PATCH /api/bookings/:id/cancel.
 - Server verifies the booking belongs to the user.
 - Updates status to “cancelled”.
 - Uses \$pull to remove the booking ID from the user’s bookings array.
 - (Optional) Could also \$inc the room’s bookingCount by -1 to keep it accurate, but not mandatory.
 - Success toast: “Booking cancelled”.
-

6. Other Requirements

- Loading Spinner – Show a centered spinner or skeleton loader whenever data is being fetched (rooms list, bookings, details page).
 - 404 Not Found Page – A custom page with a friendly message (e.g., “Page not found”) and a “Back to Home” button.
 - Dynamic Title – The browser tab title must change based on the current route. For example:
 - Home: “StudyNook – Home”
 - Rooms: “StudyNook – Available Rooms”
 - Login: “StudyNook – Login”
 - Toast Notifications – Use a library like react-hot-toast or sonner for all success/error messages. No alert() anywhere.
-

7. Challenge Requirements

7.1 JWT with HTTP-Only Cookies

- On successful login (email/password or Google), generate a JWT containing { `userId: user._id` } (optionally role).
- Store the token in an **HTTP-only cookie** (`httpOnly: true`, `secure: true` in production, `sameSite: 'strict'`).
- Create `authMiddleware` to:
 - Read token from `req.cookies.token`
 - Verify JWT

- Attach `req.user = { id: userId }`
 - Return **401 Unauthorized** if invalid or expired
 - Apply middleware to all private routes (Add Room, Book Room, My Bookings, Edit/Delete, Cancel).
 - Logout clears cookie using `res.clearCookie('token')`.
-

7.2 Search & Filter (All Rooms Page)

- Add search by room name using a search input (`/api/rooms?search=...`).
 - Use **checkbox/select filters** for amenities.
 - Optionally add filters for hourly rate or floor range.
 - Backend should use MongoDB operators like:
 - `$regex` → search by name
 - `$in` → amenities filter
 - `$gte, $lte` → optional range filtering
-

7.3 Advanced Booking Logic (Choose ANY ONE)

Implement **at least one** of the following advanced features:

- Booking conflict check using `$gte` and `$lte` to prevent overlapping bookings.

OR

- Use `$push` and `$pull` operators to manage user bookings array:
 - `$push` → add booking ID when booking is created
 - `$pull` → remove booking ID when booking is cancelled
-

8. Optional Requirements (Enhance Project)

- Dark/Light Theme Toggle – Add a switch in the navbar that toggles between dark and light themes, persisting choice via `localStorage`. Use Tailwind's dark class or a custom context.
- Framer Motion – Animate page transitions and card hover effects for a polished feel.

9. UI Design Requirements:

- **Unique Design:** First, decide what kind of website you want to make. Then, search online or check out websites like ThemeForest to get ideas for the design. But remember, your website idea shouldn't be similar to any projects you've done before or to any examples in our modules or conceptual sessions.
 - ★ You can also look for free resources on [blogs](#) to help with your website.
1. Keep the main heading style (font, size, color) consistent across all sections.
 2. Keep paragraph spacing balanced and text easily readable.
 3. Maintain uniform image sizes and spacing.
 4. Use the same button style as on the home page.
 5. Ensure good spacing and proper alignment.
 6. Navbar, Keep the heading/logo same style and size as on the home page.
 7. Use a grid layout with equal image sizes.
 8. Keep all cards equal height and width (especially in services, projects, or products section)
 9. Use the new X logo instead of the old Twitter bird to match the latest rebrand
 10. Responsiveness: Make it responsive for all devices, including mobile, tablet, and desktop views.

Resources:

- <https://uiverse.io/>
 - <https://devmeetsdevs.com/>
 - <https://bootcamp.uxdesign.cc/free-images-and-resources-collection-for-website-c77f2fc46ce5>
 - <https://themeforest.net/?srsltid=AfmBOopTj6PNz51iuV2YJXU+BP8nt19zT5LG2dToAjIHQqzNCzregn0>
 - <https://codecanyon.net/?srsltid=AfmBOooRoUfeK7lOROpchCuA4hPVj5P9WRmtDQJ9K0E6Yhf4VTrHhXKt>
-

10. What to Submit

1. Client-side code GitHub repository link (React/Vite frontend)
2. Server-side code GitHub repository link (Node/Express backend)
3. Live website URL (deployed frontend)